Reporte

Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Implementación del núcleo TechStore
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Millett, S., & Tune, N. (2015). *Patterns, principles, and practices of domain-driven design*. Wrox.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley Professional.

Equipo 6

Jesús Eduardo Moreno Osoria	**2136063**
Osvaldo Salvador Cárdenas Mejorado	**2141553**

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Implementación del Núcleo de Negocio

TechStore

Este reporte describe cómo se implementó en Python el núcleo de negocio de TechStore, una tienda ficticia de productos tecnológicos. El programa lee clientes, productos y pedidos de archivos CSV, los convierte en objetos del dominio, evalúa descuentos con Specification Pattern, controla el ciclo de vida de cada pedido con una máquina de estados que registra Domain Events y ofrece todas las operaciones mediante una API REST hecha con FastAPI. Todo funciona en memoria, sin base de datos.

La bitácora detallada de los escenarios, la explicación de los Domain Events, el diagrama de estados y la reflexión se entregan además como documentos independientes en la carpeta `docs/` del repositorio.

1. Procedimiento y desarrollo

El trabajo siguió el orden de la práctica guiada y en cada paso se ejecutaron pruebas antes de continuar: 1) dominio (Value Objects y entidades), 2) carga de los CSV, 3) Specifications, 4) máquina de estados y Domain Events, 5) escenarios E01–E15, 6) API REST y 7) mensajes para el cliente.

1.1 Organización del proyecto

Archivo	Responsabilidad
`datos/` y `cargar_csv.py`	CSV y cargador del paquete del profesor; se usan sin modificaciones.
`app/domain.py`	Money, OrderItem, Customer, Order, estados, Domain Events y excepciones de dominio.
`app/specifications.py`	Reglas de descuento (Specification Pattern) y DiscountService.
`app/services.py`	OrderService: el Application Service que coordina cada caso de uso.
`app/infrastructure.py`	Catálogo, repositorio en memoria, EventBus y EmailNotifier (correos simulados).
`app/schemas.py`, `app/api.py`	DTOs y controladores de la API REST.

Archivo	Responsabilidad
`ejecutar_escenarios.py`	Corre E01–E15 y escribe la bitácora en `evidencias/`.
`tests/`	131 pruebas automáticas con pytest.

La separación por capas permite que las reglas no dependan de la API: el mismo dominio se usa desde las pruebas, desde el programa de escenarios y desde FastAPI.

1.2 Entidades y Value Objects

Un **Value Object** se define por sus datos y no cambia después de crearse; una **entidad** tiene identidad propia y un ciclo de vida. Todas las clases validan sus reglas al crearse o al ejecutar un método, así que nunca quedan en un estado inválido.

Clase	Tipo	Reglas que protege
Money	Value Object inmutable	Importe con Decimal y moneda MXN. Rechaza float, importes negativos o no finitos; add() no mezcla monedas y devuelve un objeto nuevo.
OrderItem	Value Object inmutable	SKU obligatorio, cantidad entera de al menos 1 y precio de tipo Money. El subtotal se calcula (cantidad × precio).
Customer	Entidad	Número de cliente obligatorio; dos clientes con el mismo número son el mismo cliente.
OrderStatus	Enumeración	Los seis estados del pedido; cada uno tiene una etiqueta en español para el cliente.
Order	Entidad y Aggregate Root	Inicia en DRAFT. `status`, `items` y `events` son de solo lectura. `units` suma cantidades (no renglones); `add_item()` solo funciona en DRAFT, exige MXN y no permite pasar de 20 piezas.

Cuando una regla se rompe, el dominio lanza una excepción con nombre propio. La API la traduce a un código HTTP:

Excepción	Significado	Ejemplo real	HTTP
InvalidOrderData	Falta un dato o no se cumple una precondición	Confirmar O007 (vacío); enviar sin guía	422
InvalidOrderTransition	La operación no es posible en el estado actual	Confirmar O013 dos veces; cancelar O014 enviado	409
OrderItemLimitExceeded	El pedido pasaría de 20 piezas	Agregar 2 piezas a O009 (19 + 2)	422
NotFound	El pedido, cliente o producto no existe	Consultar O999	404

1.3 Specification Pattern

Cada condición del negocio es un objeto que responde una sola pregunta con `is_satisfied_by(pedido)`. Las condiciones se combinan con `&` (AND), `|` (OR) y `~` (NOT) sin escribir `if` anidados, y ninguna modifica el pedido: solo lo evalúan.

Specification	Pregunta
MinimumAmountSpec(500)	¿El total es estrictamente mayor que $500?
VIPCustomerSpec	¿El cliente es VIP?
MinimumUnitsSpec(4)	¿Tiene al menos 4 piezas?
ActivePromotionSpec	¿Tiene una promoción activa? (se usa negada: ~ActivePromotionSpec)

```
descuento_10 = (MinimumAmountSpec(500)      # total > $500
               & VIPCustomerSpec()          # AND cliente VIP
               & MinimumUnitsSpec(4)        # AND al menos 4 piezas
               & ~ActivePromotionSpec())    # AND NOT promoción
descuento_5  = MinimumAmountSpec(5000) & ~ActivePromotionSpec()
elegible     = descuento_10 | descuento_5   # OR; si se cumplen ambas, gana el 10 %
```

`DiscountService.evaluate()` decide el porcentaje, calcula el ahorro con centavos exactos y devuelve un mensaje para el cliente, por ejemplo: «¡Buenas noticias! Por ser cliente VIP… ahorras $210.00 MXN y pagas $1,890.00 MXN».

1.4 Máquina de estados

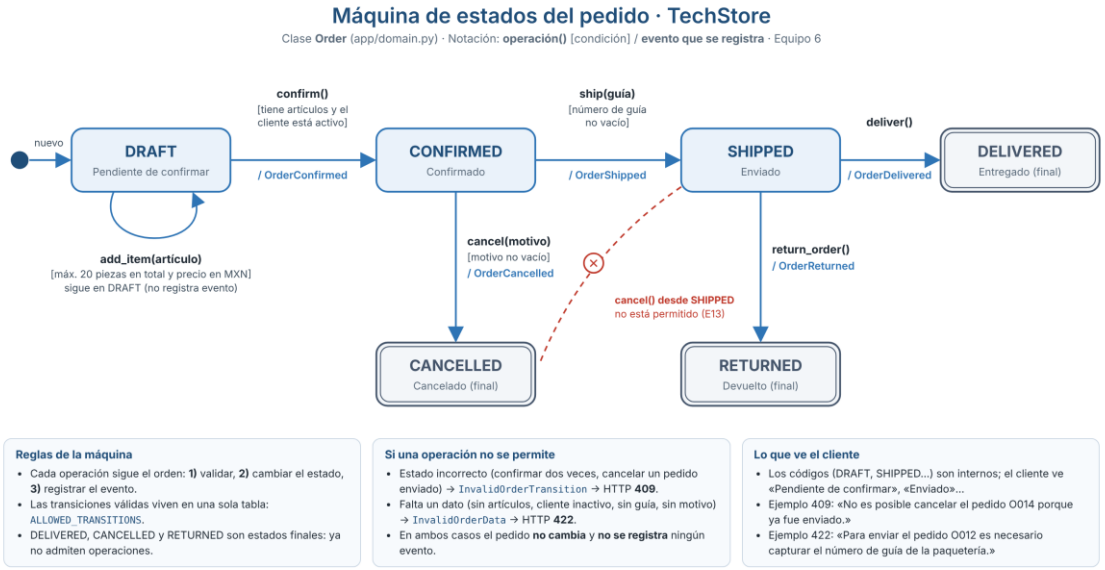


Figura 1. Máquina de estados del pedido

Solo existen cinco transiciones: confirmar (DRAFT → CONFIRMED), enviar (CONFIRMED → SHIPPED), cancelar (CONFIRMED → CANCELLED), entregar (SHIPPED → DELIVERED) y devolver (SHIPPED → RETURNED). Están en una sola tabla, `ALLOWED_TRANSITIONS`, y cada método de `Order` la consulta antes de hacer cualquier cosa. Cualquier otro salto lanza `InvalidOrderTransition` sin cambiar el pedido.

1.5 Domain Events

Cada transición exitosa registra un evento inmutable con nombre en pasado y fecha en UTC: OrderConfirmed, OrderShipped, OrderCancelled, OrderDelivered y OrderReturned. El orden dentro de cada método es siempre **validar → cambiar el estado → registrar el evento**, de modo que una operación rechazada no anuncia nada.

Después de guardar, `OrderService` publica los eventos nuevos en un `EventBus`. El suscriptor `EmailNotifier` genera el aviso para el cliente («Hola Ana Torres: confirmamos tu pedido O001 por $2,100.00 MXN…») sin que `Order` sepa que existe. Los eventos se pueden consultar en `GET /orders/{id}` y en `GET /events`.

1.6 API REST y FastAPI

Cada petición sigue el flujo **HTTP → DTO → OrderService → Order → DTO → HTTP**. Los controladores no contienen reglas: solo llaman al servicio. Un único manejador traduce las excepciones a 404, 409 o 422, y la documentación OpenAPI se genera sola en `/docs`.

Método y ruta	Uso	Respuestas
GET /orders	Listar los pedidos	200
GET /orders/{id}	Consultar un pedido con artículos, total, estado y eventos	200 · 404
POST /orders	Crear un pedido vacío	201 · 404
POST /orders/{id}/items	Agregar un producto	200 · 404 · 409 · 422
POST /orders/{id}/confirm	Confirmar	200 · 404 · 409 · 422
POST /orders/{id}/ship	Enviar con número de guía	200 · 404 · 409 · 422
POST /orders/{id}/cancel	Cancelar con motivo	200 · 404 · 409 · 422
POST /orders/{id}/deliver	Marcar como entregado	200 · 404 · 409

Método y ruta	Uso	Respuestas
POST /orders/{id}/return	Registrar devolución	200 · 404 · 409
GET /orders/{id}/discount	Evaluar el descuento	200 · 404
GET /events · GET /notifications	Eventos publicados y avisos generados	200
POST /admin/reset	Volver a cargar los 16 pedidos del CSV	200

1.7 Mensajes para el cliente

Siguiendo la recomendación del profesor, los mensajes ya no muestran códigos internos. El programa sigue usando DRAFT, CONFIRMED, etc., pero al cliente le responde en español sencillo:

Situación	Antes	Ahora
Confirmar dos veces	No se puede confirmar un pedido en estado CONFIRMED	El pedido O013 ya fue confirmado; no es necesario confirmarlo otra vez.
Cancelar un pedido enviado	No se puede cancelar un pedido en estado SHIPPED	No es posible cancelar el pedido O014 porque ya fue enviado.
Dato con formato incorrecto	Input should be a valid integer, unable to parse string as an integer	Revisa la información enviada: 'quantity' debe ser un número entero.

Cada error responde `{"error": código, "mensaje": texto}`; cada pedido trae `status_label` («Pendiente de confirmar», «Enviado»…) y cada evento una `description` («Pedido confirmado»). La prueba `test_mensajes_de_error_sin_codigos_internos` provoca todos los tipos de error y falla si algún mensaje contiene un código interno.

2. Ejemplos de funcionamiento

2.1 Pedido O001: del CSV al descuento del 10 %

En `pedidos.csv`, O001 pertenece a C001 (Ana Torres, VIP y activa) y no tiene promoción. En `detalle_pedidos.csv` tiene 2 × SKU001 ($600.00) y 2 × SKU002 ($450.00). Al cargarlo, `Order` calcula `units` = 2 + 2 = **4** y `total` = 1,200.00 + 900.00 = **$2,100.00**.

Condición	Valor de O001	¿Se cumple?
Cliente VIP	C001 es VIP	Sí
Total > $500	$2,100.00	Sí
Al menos 4 piezas	4	Sí
NOT promoción activa	sin promoción	Sí
Total > $5,000 (regla del 5 %)	$2,100.00	No

Como la regla del 10 % se cumple completa, gana sobre la del 5 %: descuento = 10 % de $2,100.00 = **$210.00** y total a pagar = **$1,890.00**. Evaluar el descuento no modifica el pedido: `order.total` sigue siendo $2,100.00.

2.2 Pedido O009: el límite de 20 piezas

O009 tiene un solo renglón con 19 piezas de SKU003. Al intentar agregar 2 más, `add_item()` calcula 19 + 2 = 21 **antes** de tocar la lista de artículos y lanza `OrderItemLimitExceeded`. El pedido conserva sus 19 piezas y la API responde 422 con el mensaje «…ya tiene 19 y se intentaron agregar 2; puedes agregar como máximo 1 más». Este caso muestra por qué se suman cantidades y no se cuentan renglones: O009 tiene un solo renglón, pero 19 piezas.

3. Pruebas y validaciones

3.1 Resultados de la ejecución

Comando	Resultado
`python cargar_csv.py`	16 pedidos en DRAFT; O001 con 4 piezas y 2100.00
`python -m pytest`	131 pruebas aprobadas, 0 fallos
`python ejecutar_escenarios.py`	15/15 escenarios con el resultado esperado
`uvicorn app.api:app --reload`	API disponible; `/docs` y `/openapi.json` responden

3.2 Qué verifican las pruebas

Archivo	Pruebas	Qué verifica
test_domain.py	68	Money, OrderItem, Customer, invariantes de Order, las 5 transiciones válidas, las 25 inválidas y que sus mensajes no muestren códigos internos
test_specifications.py	22	Tablas de verdad de AND, OR y NOT; cada regla atómica (500.00 vs 500.01, 3 vs 4 piezas); descuentos y prioridad del 10 %
test_escenarios.py	16	Una prueba por escenario E01–E15 y la carga correcta de los CSV
test_api.py	25	Códigos 200, 201, 404, 409 y 422; mensajes amigables; eventos publicados; avisos al cliente; reinicio y contrato OpenAPI

3.3 Verificaciones adicionales

Para comprobar que las pruebas sí detectan errores, se introdujeron a propósito seis fallas en una copia del código y se ejecutó pytest con cada una:

Falla introducida a propósito	Pruebas que fallaron
Umbral >= en lugar de > en MinimumAmountSpec	1
Contar renglones en lugar de sumar piezas	17
Permitir cancelar desde SHIPPED	5
Dar prioridad al 5 % sobre el 10 %	1
Ignorar que el cliente esté inactivo	4
Responder 422 en lugar de 409 en transiciones inválidas	2

Las seis fallas fueron detectadas. Además, el proyecto se ejecutó desde un ZIP limpio (sin cachés) y el dominio con los escenarios también se probó con Python 3.11, la versión mínima que pide la práctica.

4. Bitácora de escenarios E01–E15

Cada escenario se ejecutó con datos recién cargados y sin modificar los CSV. Resultado: **15 de 15 escenarios con el resultado esperado**. La versión completa, con precondiciones, mensajes exactos y explicación de cada caso, está en el documento «Bitácora de escenarios».

Caso	Pedido y precondiciones	Operación	Resultado real	Evento(s)
E01	O001 · C001 VIP, activo · 4 pzas · $2,100.00 · sin promoción	Evaluar descuento	**10 %**. Descuento $210.00; total a pagar $1,890.00. → DRAFT	Ninguno (evaluar no cambia el pedido)
E02	O002 · C002 no VIP, activo · 2 pzas · $1,050.00 · sin promoción	Evaluar descuento	**0 %**. Total a pagar $1,050.00. → DRAFT	Ninguno
E03	O003 · C003 VIP, **inactivo** · 4 pzas · $2,100.00	Confirmar	**Rechazado** con InvalidOrderData → DRAFT	Ninguno
E04	O005 · C005 VIP, activo · 4 pzas · $2,100.00 · **con promoción**	Evaluar descuento	**0 %**. Total a pagar $2,100.00. → DRAFT	Ninguno
E05	O006 · C006 no VIP, activo · 1 pza · $18,000.00 · sin promoción	Evaluar descuento	**5 %**. Descuento $900.00; total a pagar $17,100.00. → DRAFT	Ninguno
E06	O007 · C007 VIP, activo · **0 pzas** · $0.00	Confirmar	**Rechazado** con InvalidOrderData → DRAFT	Ninguno
E07	O008 · C008 no VIP, activo · **20 pzas** · $2,400.00	Confirmar	**Éxito**: pasa a CONFIRMED («Confirmado»). → CONFIRMED	OrderConfirmed

Caso	Pedido y precondiciones	Operación	Resultado real	Evento(s)
E08	O009 · C009 VIP, activo · 19 pzas · $2,280.00	Agregar SKU003 × 2	**Rechazado** con OrderItemLimitExceeded → DRAFT (sigue con 19)	Ninguno
E09	O010 · C010 no VIP, activo · 2 pzas · **$500.00**	Evaluar descuento	**0 %**. Total a pagar $500.00. → DRAFT	Ninguno
E10	O011 · C011 VIP, activo · **3 pzas** · $1,500.00	Evaluar descuento	**0 %**. Total a pagar $1,500.00. → DRAFT	Ninguno
E11	O012 · C012 no VIP, activo · 1 pza · $850.00	Confirmar y enviar sin guía	Confirmar: **éxito**. Enviar: **rechazado** con InvalidOrderData → CONFIRMED	OrderConfirmed
E12	O013 · C001 VIP, activo · 1 pza · $600.00	Confirmar dos veces	Primera: **éxito**. Segunda: **rechazada** con InvalidOrderTransition → CONFIRMED	OrderConfirmed (solo uno)
E13	O014 · C002 no VIP, activo · 1 pza · $600.00	Confirmar, enviar y cancelar	Confirmar y enviar: **éxito**. Cancelar: **rechazado** con InvalidOrderTransition → SHIPPED	OrderConfirmed, OrderShipped
E14	O015 · C005 VIP, activo · 1 pza · $450.00	Confirmar, enviar y devolver	**Éxito**: termina en RETURNED («Devuelto»). → RETURNED	OrderConfirmed, OrderShipped, OrderReturned
E15	O016 · C007 VIP, activo · 1 pza · $2,100.00	Confirmar, enviar y entregar	**Éxito**: termina en DELIVERED («Entregado»). → DELIVERED	OrderConfirmed, OrderShipped, OrderDelivered

5. Evidencias

5.1 Carga de los 16 pedidos

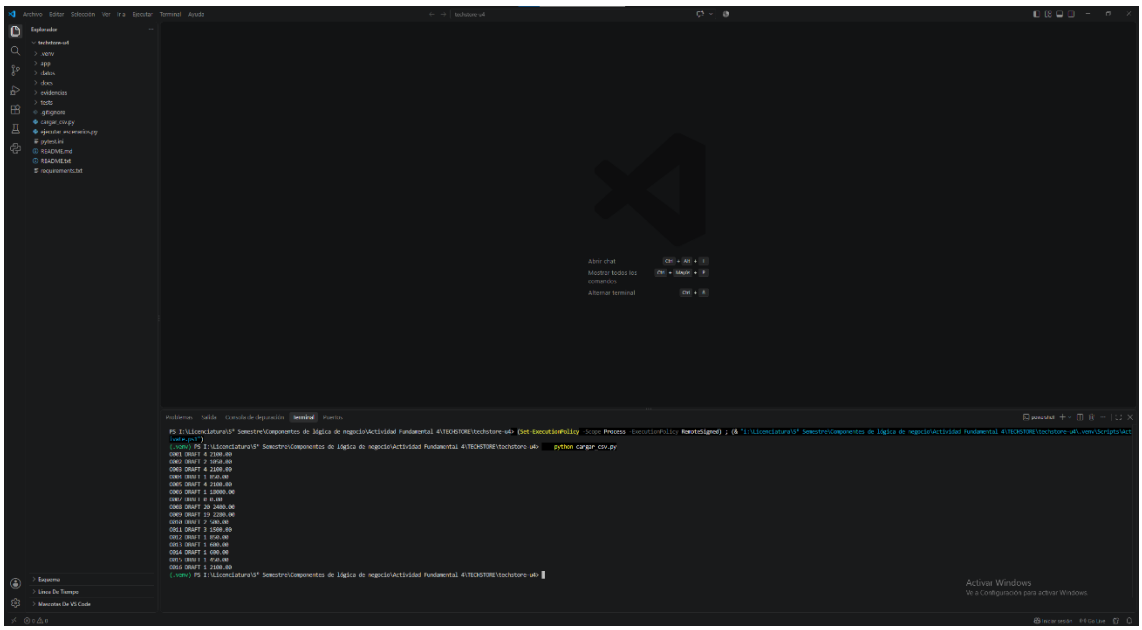


Figura 2. Ejecución de python cargar_csv.py: los 16 pedidos inician en DRAFT.

5.2 Pruebas automatizadas

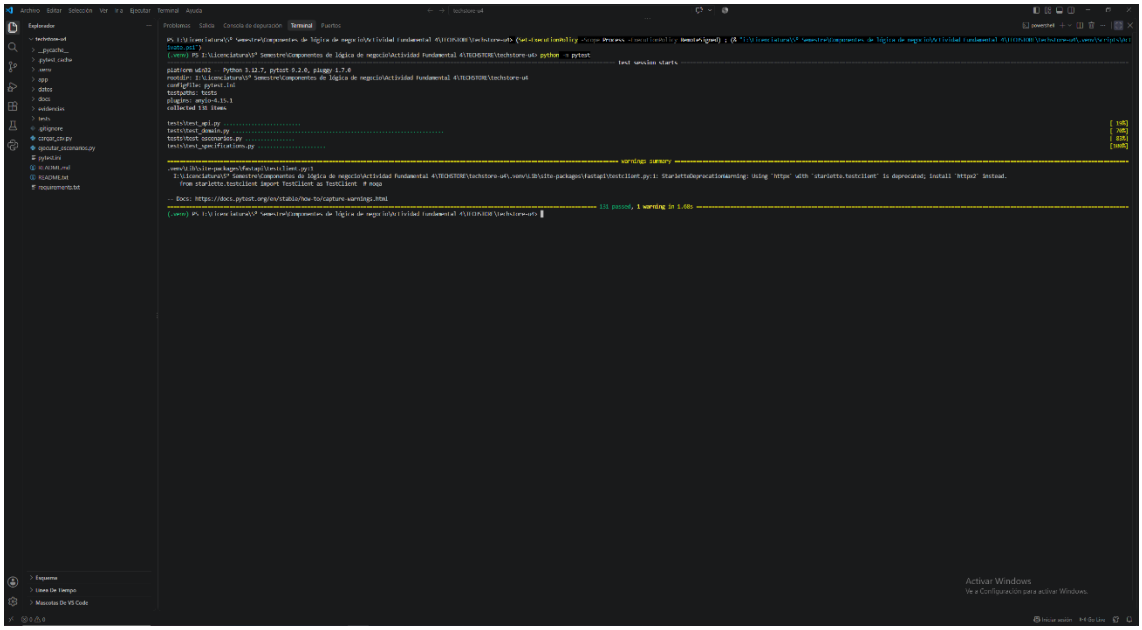


Figura 3. Ejecución de python -m pytest: 131 pruebas aprobadas.

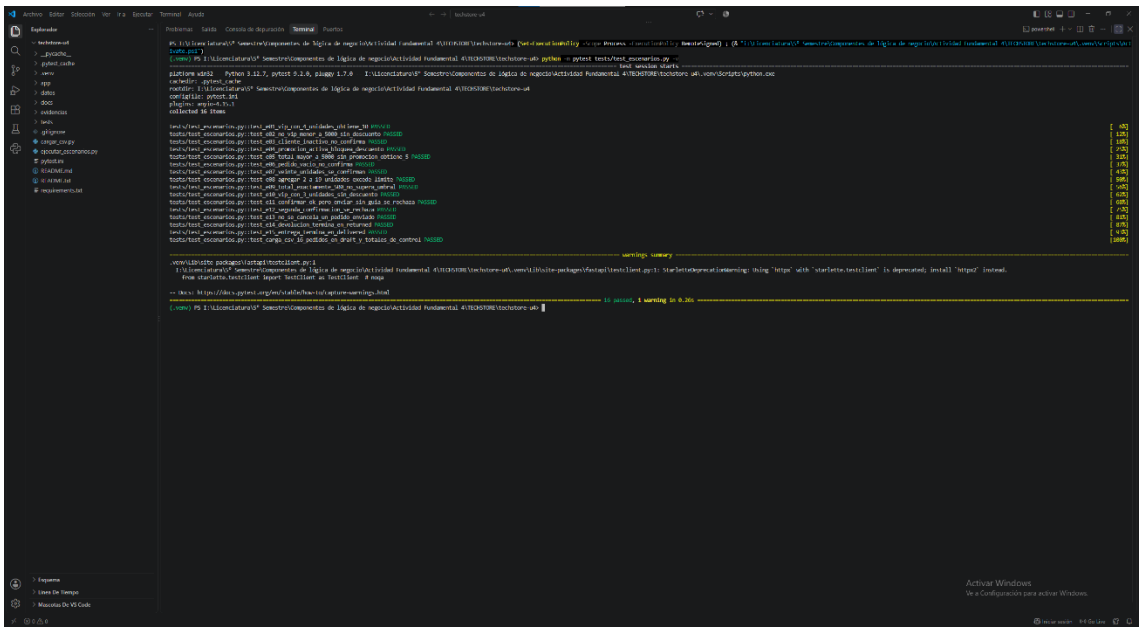


Figura 4. Pruebas de los escenarios en modo detallado: cada escenario aparece como PASSED.

5.3 Ejecución de los escenarios E01–E15

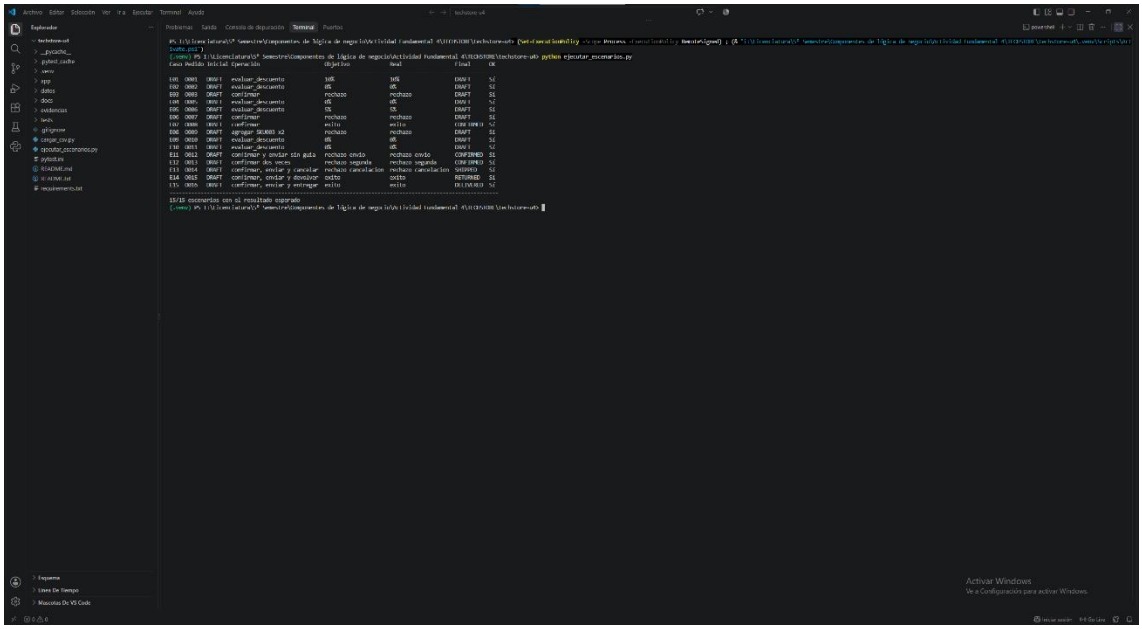


Figura 5. Ejecución de python ejecutar_escenarios.py: 15/15 escenarios con el resultado esperado.

5.4 Documentación de la API

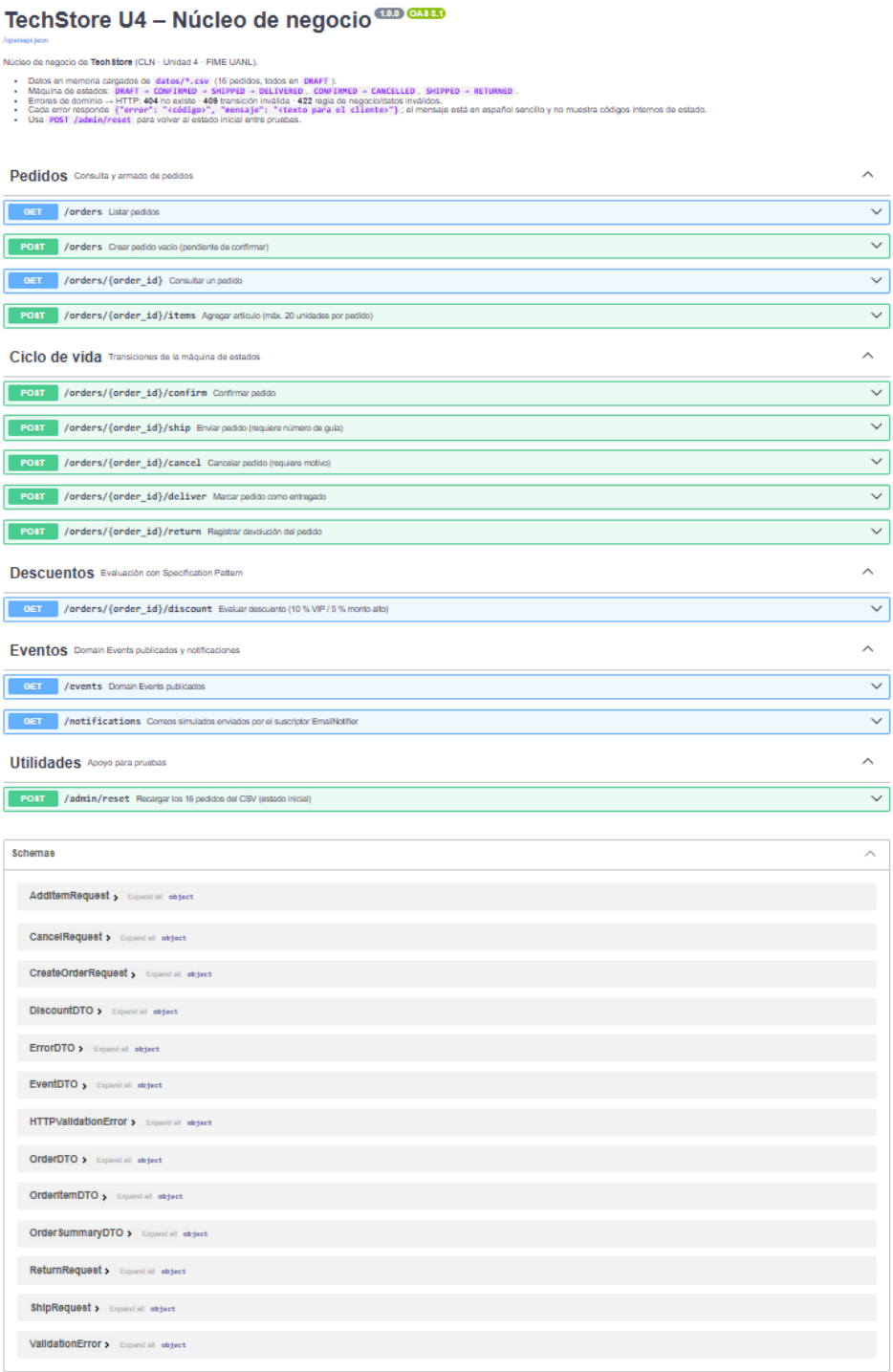


Figura 6. Swagger UI en /docs con los endpoints agrupados por tema.

5.5 Operación exitosa

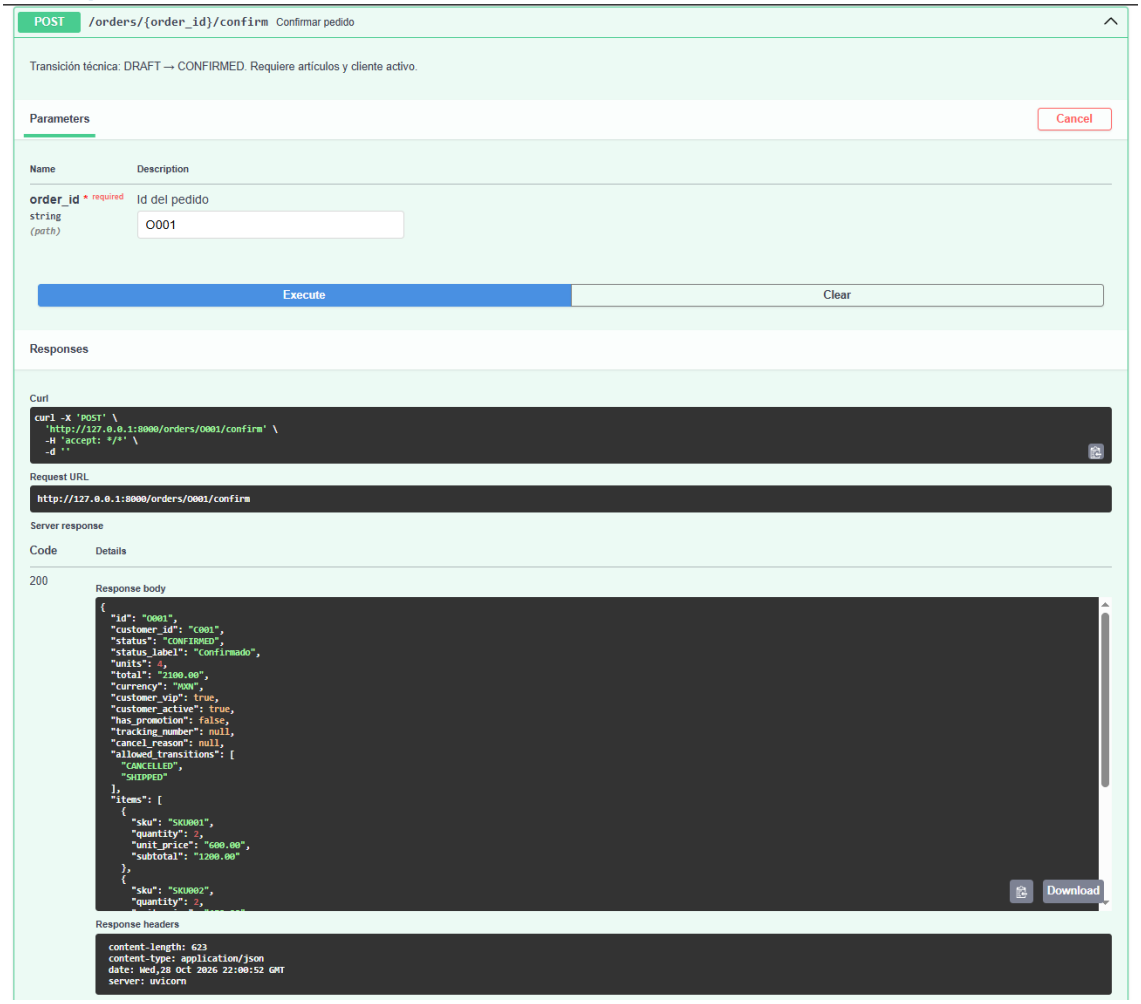


Figura 7. Confirmar O001 responde 200: el pedido queda «Confirmado» y registra OrderConfirmed

5.6 Operaciones rechazadas

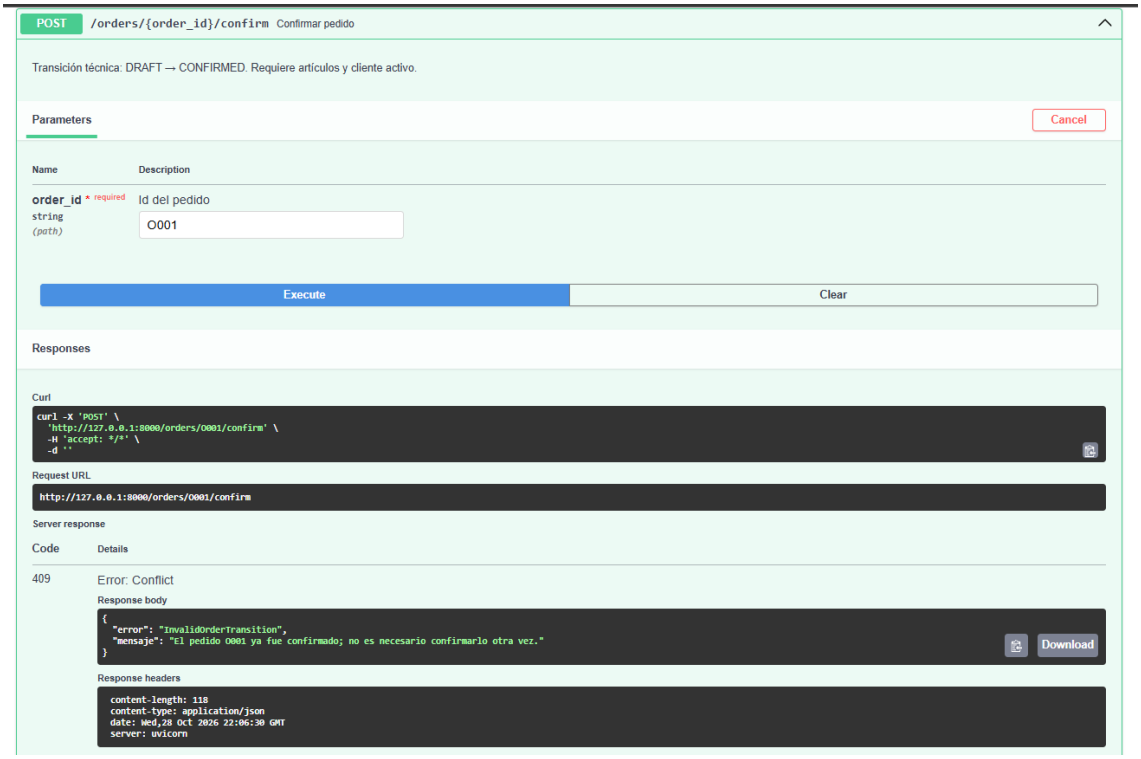


Figura 8. Confirmar O001 por segunda vez responde 409 (InvalidOrderTransition)

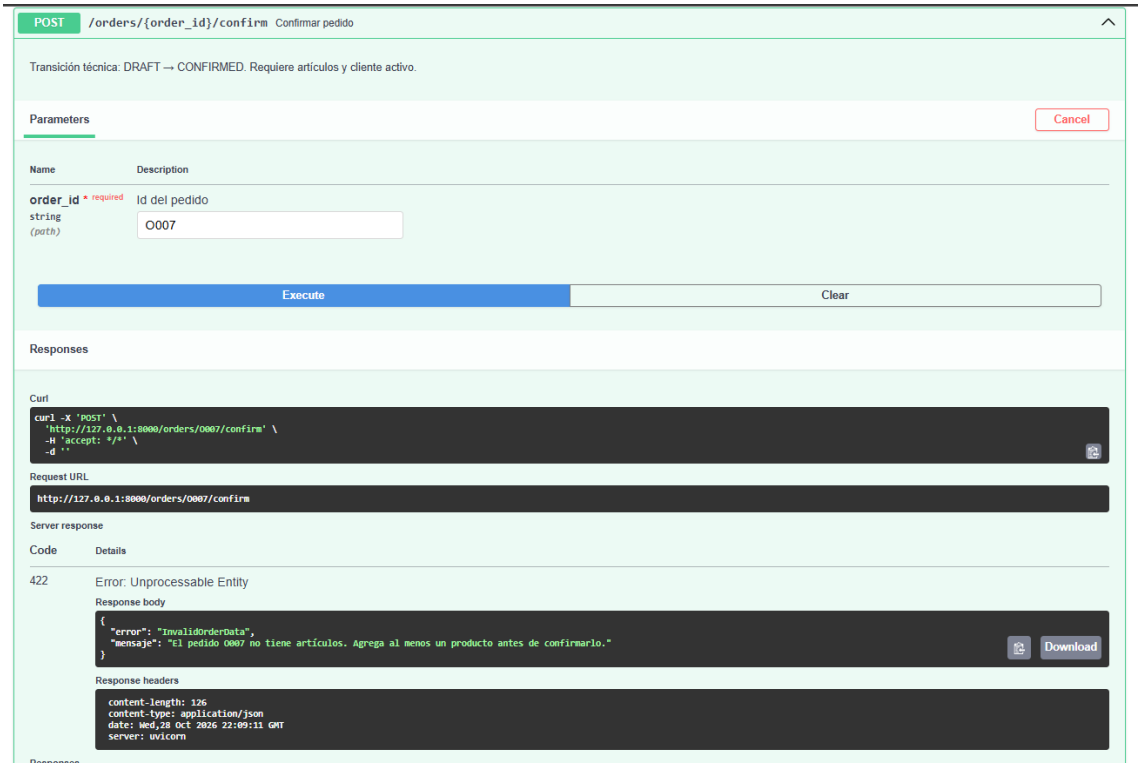


Figura 9. Confirmar O007, que no tiene artículos, responde 422 (InvalidOrderData).

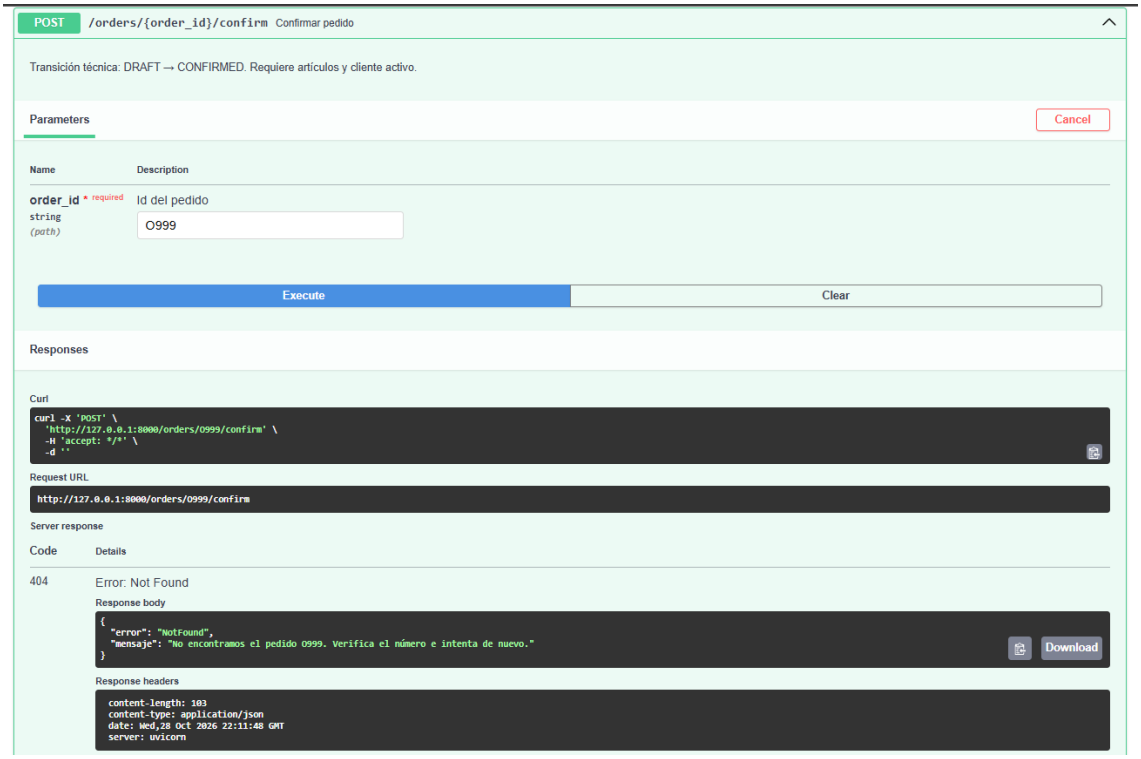


Figura 10. Consultar O999, que no existe, responde 404 (NotFound).

6. Reflexión: errores encontrados

Estos son los tres errores más importantes que encontramos; el detalle de cada uno (cómo se detectó, causa, solución y comprobación) está en el documento «Reflexión».

Error	Solución	Aprendizaje
El estado del pedido y su lista de artículos se podían modificar desde fuera de Order	Atributos privados con propiedades de solo lectura; los artículos se devuelven como tupla	El Aggregate Root debe ser la única puerta para cambiar sus datos
Los mensajes mostraban códigos internos (DRAFT, CONFIRMED) y errores de formato en inglés	Etiquetas en español, mensajes según la situación, traductor de errores de validación y una prueba que lo vigila	El código interno es para el programa; el mensaje es para la persona
El total de un pedido vacío aparecía como 0 y no como 0.00	Crear el cero como Decimal("0.00")	Con dinero importan el valor y la precisión

7. Preguntas del reporte

¿Por qué se usa Decimal en lugar de float para precios?

Porque `float` guarda los números en binario y muchos decimales no se pueden representar exactamente: `0.1 + 0.2` da `0.30000000000000004`. En dinero esos pequeños errores se acumulan y terminan en centavos de diferencia. `Decimal` trabaja en base 10, es exacto y permite elegir cómo redondear; por eso `Money` rechaza `float` y los precios se leen del CSV como `Decimal("600.00")`.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es un producto distinto dentro del pedido; las unidades son la suma de las cantidades de todos los renglones. O001 tiene 2 renglones y 4 piezas; O008 tiene 1 renglón y 20 piezas. Si el límite contara renglones, alguien podría pedir un solo renglón con 500 piezas. Cuando cambiamos `units` a propósito para contar renglones, fallaron 17 pruebas.

¿Qué ventaja tiene encapsular una regla en una Specification?

La regla tiene un nombre que se entiende, se prueba sola y se reutiliza desde la API, los escenarios o un proceso batch sin copiar condiciones. Se combina con AND, OR y NOT para formar reglas nuevas, y si cambia un umbral se corrige en un solo lugar. Además permite explicar qué falló: en E10 la única condición que no se cumple es «al menos 4 piezas».

¿Por qué los eventos se registran solo después de una transición válida?

Porque un evento anuncia algo que ya ocurrió. Si se registrara antes de validar y la operación fallara, los suscriptores (correo, inventario, facturación) reaccionarían a un hecho falso; por ejemplo, el cliente recibiría «tu pedido va en camino» de un pedido que nunca se envió. En E11 y E12 la operación rechazada no deja ningún evento.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Todo excepto `app/api.py` y `app/schemas.py`: el dominio, las Specifications, el servicio de aplicación y la infraestructura en memoria. `ejecutar_escenarios.py` ya lo demuestra: es una segunda interfaz, de consola, que usa exactamente el mismo dominio sin servidor web.

8. Conclusión general

La actividad nos permitió pasar del diseño de la Unidad 3 a código que funciona y que se puede comprobar. Los Value Objects evitan datos inválidos desde que se crean, las Specifications vuelven legibles y reutilizables las políticas de descuento, la máquina de estados impide los saltos que el negocio no permite y los Domain Events comunican lo que ocurrió sin acoplar las piezas. Como la API solo es una puerta de entrada, el mismo núcleo funciona desde las pruebas, los escenarios o cualquier otra interfaz. Por último, la revisión de los mensajes nos recordó que un sistema no solo debe ser correcto: también debe hablarle claro a quien lo usa.

9. Conclusiones individuales

Jesús Eduardo Moreno Osoria

Lo que más me ayudó fue entender la diferencia entre una entidad y un Value Object: Money no necesita identidad, pero un pedido sí. Ver que el límite de 20 piezas vive dentro de `Order` y no en la API me hizo comprender por qué la regla se cumple sin importar desde dónde se use el programa.

Osvaldo Salvador Cárdenas Mejorado

Aprendí que una máquina de estados hace explícito qué puede pasar con un pedido y qué no. Las pruebas de transiciones inválidas, y el ejercicio de romper el código a propósito para ver que las pruebas lo detectaban, me dieron confianza en que el sistema realmente protege las reglas del negocio.

9. Enlace del repositorio en GitHub

https://github.com/OsvaldoCMX/CNLEG_003_Equipo6_Unidad4